

Pandas & scikit-learn Patterns You'll See This Course — Session 7

Machine Learning in Geoscience — pre-session reference · read before Session 7

Session 7 doesn't introduce new modeling ideas — it assembles Sessions 1-6 into one pipeline, picks a final model per horizon, and opens the test block exactly once. The new patterns here are about *choosing* between already-computed results (sorting, tie-breaking, binning) and about proving a model is reproducible without writing anything to disk.

1. Ranking and tie-breaking candidates

```
sorted(candidates, key=lambda k: val[k]['metrics']['MAE'])
```

Sorts a list of names by looking up a nested value for each one — here, "order the model names by their validation MAE," lowest first. The `key=` function is called once per item; it doesn't need to return what gets sorted, just what to sort by.

```
min(tied, key=lambda k: SIMPLE_ORDER.index(k))
```

Among several models judged "close enough" (within `TIE_TOLERANCE`), picks the simplest one by looking up each name's position in a fixed preference list. `.index(k)` gives that position; the smallest position wins. This is how the project prefers persistence over a random forest when the two are statistically indistinguishable.

```
SIMPLE_ORDER = ['Persistence', 'Diurnal climatology', 'Random Forest', 'Boosting']
tied = [k for k in order if val[k]['MAE'] - val[best]['MAE'] <= TIE_TOLERANCE]
chosen = min(tied, key=lambda k: SIMPLE_ORDER.index(k)) # simplest of the tied models
```

2. Binning a continuous variable

```
pd.cut(series, bins=(0,1,2,4,8,100), labels=('0-1','1-2','2-4','4-8','>8'))
```

Turns a continuous column (observed wind speed) into a categorical one (a wind-speed "regime") by cutting it at fixed thresholds. Combine with `.groupby('regime')` to get error broken down by calm/moderate/stormy conditions instead of one overall average.

Before Session 7 — a 10-minute self-check

- Sort a small list of dict keys by a value inside each dict using `sorted(..., key=lambda k: d[k])`.
- Try `pd.cut()` on a toy numeric column with your own bin edges and labels, then `.groupby()` the result.
- If something above is still unclear, that's fine — flag it at the start of the session instead of getting stuck mid-lecture.

Machine Learning in Geoscience — Session 7 pre-read. This sheet covers recognition only; the course notebook is where these patterns get used for real, with full explanations in context.

3. Serializing a model without writing a file

```
io.BytesIO() + joblib.dump / load
```

`joblib.dump(model, path)` normally writes a file to disk; this course never does. Writing to an in-memory `BytesIO` buffer instead proves the same thing — that a model can be serialized and reloaded to give identical predictions — without touching the filesystem.

```
buffer = io.BytesIO()
joblib.dump(model, buffer)
buffer.seek(0) # rewind before reading back
reloaded = joblib.load(buffer)
assert np.allclose(model.predict(X), reloaded.predict(X))
```

4. Refitting on train + validation for the final model

```
pd.concat([X[cols].iloc[tr], X[cols].iloc[va]])
```

Once a model is *selected* using the validation block, the final deployed version is refit on train+validation combined (more data), and evaluated on the untouched test block. Stacking two row-blocks with `pd.concat` is how that combined training set is built — selection and final-fit data must stay separate steps.

5. A guard flag around code you must run at most once

```
if OPEN_TEST_BLOCK: ...
```

Not new syntax, but a new *discipline*: everything that touches the test block sits behind one boolean flag, set once, so that "did I look at the test score before freezing my model choice?" has an honest, visible answer instead of being buried in cell-execution order.